

One Namespace, or One **Vote** Each

Nine patterns measured on six topologies in nine runnable shapes. Every card is a drawing and a decision: what the shape is, what it buys, what it costs, and the check ID that proves it. One whole card is things this lab got wrong and corrected.

THE QUESTION EVERY CARD SERVES

Two regions, one platform. **Do you want one global stream name, or do you want each region to keep working when the other goes dark?** No shape in this lab gives you both.

THE RULE THAT ORDERS THE DECK

A **gateway** fuses two regions into one JetStream system. A **leaf link** gives each one its own. An **account** is the only real wall. A **domain** is a namespace, not a wall. Every card is a consequence of those four sentences.

CARD FAMILIES

■ Links — gateway, leaf node

■ Quorum — arbiters, and what a write is

■ Traps — both links at once

■ Boundaries — account, domain

■ Copying — mirrors, export / import

■ Retractions — what we got wrong

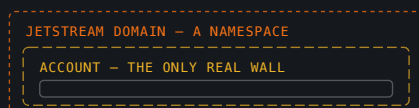


KEY

Every card is drawn to the report's rules

The figures on these cards use the **same drawing grammar** as [REPORT.html](#). A boundary means the same thing in both. If a card and the report disagree, the report is right — it is generated from the run.

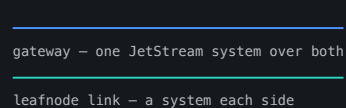
BOUNDARIES — THE THREE WALLS, AND WHICH OF THEM IS REAL



orange dashed — jetstream.domain. Names one JetStream system: one meta group, one set of stream names. Data crosses it when you name the other domain's API.



yellow dashed — an account. Two accounts cannot see each other's subjects, streams or consumers at all. A plain box inside = a cluster; a red node is frozen.



mirror — a copy somebody wrote down
kill -STOP — how every outage here is made

Each figure ends with the check IDs it draws. A claim with no ID on the drawing is not measured on that rig.

TWO THINGS THE DRAWINGS WILL NOT LET YOU ASSUME

- › **A domain box is not an account box.** They are deliberately different colours because they are not the same kind of boundary. A domain separates *names*. An account separates *data*, and nothing crosses it without an export.
- › **A frozen region is `kill -STOP`, not a severed link.** Both sides of a real WAN partition stay alive and cannot reach each other. That rig does not exist here, and no figure in this deck draws one.

01

LINKS — T2 / A

A gateway buys one name and sells your survival

Join two regions with a **gateway** and they stop being two JetStream systems. They become one. One meta group, one set of stream names, one majority — held across the sea.

DECISION

Use a gateway when you want **one global stream namespace** and you can put a third site in. Do not use a plain two-region gateway if either region must survive the other going dark.

THE SHAPE, AND WHAT IT DOES WHEN AU GOES DARK

JETSTREAM DOMAIN — NONE SET — ONE SYSTEM OVER ALL SIX SERVERS

ACCOUNT LB — ONE STREAM NAMESPACE FOR BOTH REGIONS

CLUSTER za — REGION za

t-za-1
127.0.0.1:4231

t-za-2
127.0.0.1:4232

t-za-3
127.0.0.1:4233

gateway

CLUSTER au — REGION au — FROZEN, kill -STOP

t-au-1
127.0.0.1:4234

t-au-2
127.0.0.1:4235

t-au-3
127.0.0.1:4236

healthy — every server names the same meta leader

healthy — ZA asks for a stream name AU already owns

AU dark — create any stream, from the live region

AU dark — publish into a stream that already lives in ZA

AU dark — read AU's own stream from ZA

yes — A3 A4 A5

refused, 10058 — A6 A7 A8

fails, 10008 JSClusterNotAvailErr — A11

1 message stored — A12

fails — A21

CONTINUES

The drawing is the whole cost: **one account box, one domain box, six servers, one vote**. Half the vote is in the other region.

Card 01 · 1 of 2

MECHANISM

- › A gateway joins **clusters**, not systems. With no `jetstream.domain` set, the six servers elect **one** meta group across both regions.
- › The meta group is what creates, updates and deletes streams. It needs a majority of all six. Three dark servers is not a majority of six.
- › Stream **replicas** are placed inside one cluster. They keep their own majority when the other region dies — which is why publishing still works while creating does not.
- › One namespace is real and enforced: ZA cannot create a stream with a name AU already owns (`10058` , `A6` – `A8`).

PRO

- › **One name means one thing everywhere.** No prefixes, no per-region registry, no chance of two different `ORDERS` streams.
- › Subject interest crosses the gateway on its own. No mirror to configure.
- › Placement by tag lets you say where a stream physically lives while keeping one catalogue.

CON

- › **Two regions cannot hold a majority of two regions.** Lose one and the whole system loses the ability to change itself.
- › The failure is silent to a plain publisher. `nats pub` is a core publish and exits 0 whether the message was stored or not.
- › You cannot read the dark region's data at all (`A21`). Its replicas are all in the dark region.

WHAT THE DRAWING DOES NOT SHOW

- › This is a **frozen region**, not a split brain. Both sides of a real WAN partition stay alive and cannot see each other. That rig is not built here, so this deck makes no claim about it.

VERDICT

A two-region gateway is a **single system wearing two coats**. Take it only with a third site (card 04), or do not take it.

02

LINKS — T5 / D

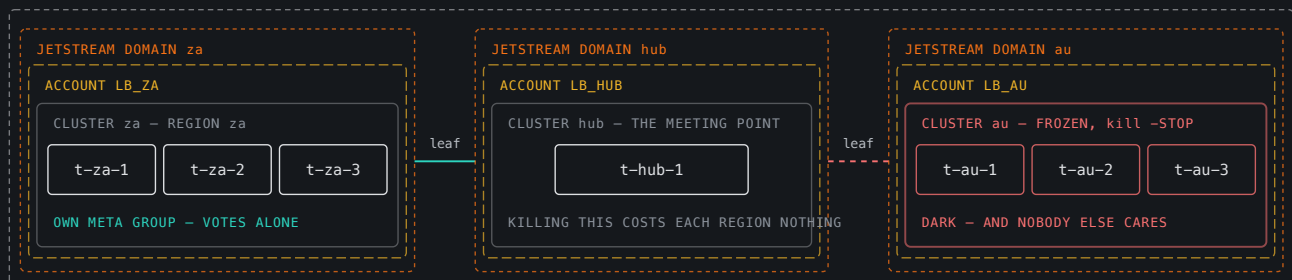
A leaf link buys a vote each and sells you a double copy

Give each region its own `jetstream.domain` and hang it off a small hub on a **leafnode link**. Now there are three JetStream systems, not one. Each region votes alone.

DECISION

Use a leaf link when **each region must keep working alone**. Accept that a stream name no longer means one thing, and that any shared data must be copied on purpose.

THE SHAPE, AND WHAT IT DOES WHEN AU GOES DARK



healthy - each region names its own meta leader
 AU dark - ZA creates a brand new stream
 AU dark - ZA publishes into that stream
 the same stream name may now exist once per domain
 so a shared copy must be written down on purpose

three systems, three votes - D24
 works - D25
 message stored - D26
 nobody can refuse the overlap - D21 D22
 mirror, naming the other API - D8

MECHANISM

- › Setting `jetstream.domain` per cluster makes each cluster **its own JetStream system**, with its own meta group and its own stream names.
- › A leafnode link carries **subjects** between systems. It does not merge them, and it does not lend a vote.
- › Each region's meta group is three servers inside one region. A majority of three is two, and both are at home. The other region going dark changes nothing.
- › The hub is only a meeting point. The lab kills the hub outright and each region carries on.

PRO

- › **Each region survives alone.** Create, publish and consume all keep working (`D25` , `D26`).
- › Failure is contained by design. There is no shared majority to lose.
- › Regions can be added or removed one at a time. Nothing re-votes.

CON

- › **A stream name means nothing on its own any more.** `ORDERS` in `za` and `ORDERS` in `au` are two unrelated streams (`D21` , `D22`).
- › Nothing is shared until somebody writes a mirror. That mirror is a real thing to build, watch and pay for.
- › Two copies means two bills and two truths that can drift.

VERDICT

This is the shape to start from for two real regions. Take the double copy on purpose, rather than take an outage by accident.

03

BOUNDARIES — ACCOUNTS VS DOMAINS

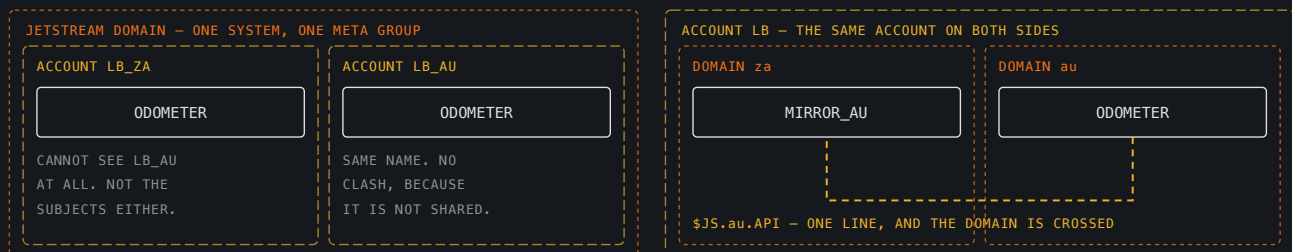
The account is the wall. The domain is only a namespace.

They look alike on a diagram and they are not alike at all. One of them stops data. The other only stops a *name* from resolving, and it can be talked around in one line.

DECISION

Put a **tenant, a customer or a blast radius** behind an account. Use a domain to give a region its own JetStream system. **Never use a domain as a security boundary.**

THE SAME TWO STREAMS, DRAWN BOTH WAYS



THE ACCOUNT IS THE WALL

two accounts, one system: both hold a stream ODOMETER
 publish once in LB_ZA, then count in both accounts
 ask across a domain by stream name alone
 ask across a domain by naming its API

THE DOMAIN IS ONLY A NAMESPACE

each lands in its own — D17
 1 and 0 — nothing leaked — D18
 fails — H7
 data crosses — D8 H11

MECHANISM

- › An **account** is a whole separate subject space. Two accounts share a server and share nothing else — not subjects, not streams, not consumers.
- › The only way out of an account is an explicit **export** that the other side must explicitly **import**. Both halves are written by a human. See card 07.
- › A **domain** names one JetStream system. A bare stream name resolves only inside the asking domain, which is why **H7** fails.
- › Name the other domain's API — `mirror.external.api = $JS.au.API` — and the data crosses. That is not a hole; it is the feature. But it is **not a wall**.

PRO — USE AN ACCOUNT WHEN

- › You need a **tenant** to be unable to see another tenant, by construction.
- › You want a blast radius: a bad consumer in one account cannot touch another.
- › You want sharing to be a deliberate, reviewable act.

CON — THE COST OF AN ACCOUNT

- › Every crossing must be built. Nothing is shared by default, ever.
- › An export / import pair is two edits in two places. Get one wrong and it silently does nothing.
- › Tooling and credentials multiply. One context per account.

THE MISTAKE THIS CARD EXISTS TO STOP

- › **"We gave each region its own domain, so regions are isolated."** They are not. A domain separates names. Anyone who can reach the server and holds the account's credentials can name the other domain's API and read it — exactly what **D8** and **H11** do on purpose.

VERDICT

Accounts for who may see what. Domains for who votes with whom. They are not interchangeable, and swapping them is a security bug.

04

QUORUM — T4 / F

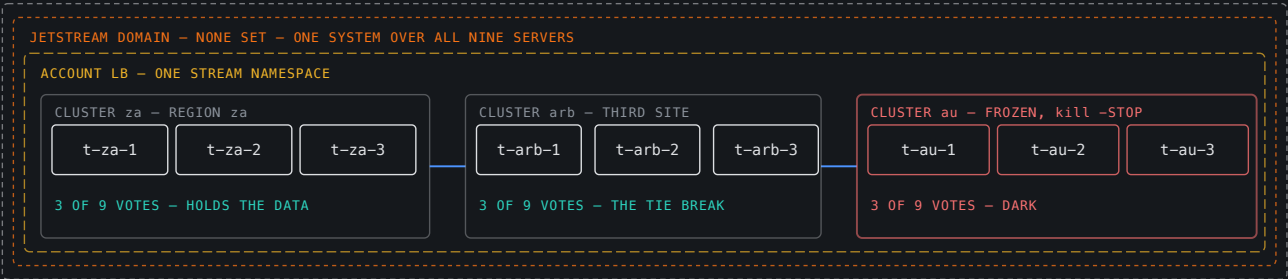
An arbiter site buys the vote back

Two regions cannot hold a majority of two regions. **Three sites can.** Add a third cluster that holds almost nothing and votes, and the gateway shape stops being fragile.

DECISION

If you want one global namespace **and** survival, you must pay for a third site. There is no two-site version of this. A one-node arbiter works; a three-node arbiter is the safe one.

NINE SERVERS, THREE SITES, ONE VOTE — AND AU GOES DARK



healthy - nine servers name one meta leader
AU dark - create a stream, tried from both live sites
AU dark - publish into a stream that lives in ZA
that stream keeps all three of its own replicas inside ZA

yes - F3 F4
works, 6 of 9 still vote - F4 F23
message stored - F21 F24
which is why it keeps working - F9 F21

MECHANISM

- › The meta group counts **servers**, not sites. Nine servers need five to agree. Two live sites give six. Six beats five.
- › The arbiter needs to be reachable and to vote. It does not need to hold your data, so it can be small and cheap.
- › The lab measured **both** sizes: a one-node arbiter (**T3 / C**) and a three-node arbiter (**T4 / F**). Both restore stream creation with a region dark.
- › A one-node arbiter restores the vote and **becomes the single point of failure**. Lose it and a region, and you are back to card 01.

PRO

- › **One namespace and survival, together.** The only shape in this lab that gives both.
- › The third site is small. It is a cost, not a second data centre.
- › Recovery is automatic. Nothing is promoted by hand.

CON

- › **You must really have a third place.** Three availability zones in one region are not three sites for a regional outage.
- › Latency: the meta group now spans the ocean. Every stream change waits for it.
- › It is still one system. One bad operation can still affect both regions.

HOW TO SIZE THE ARBITER

- › Three nodes if you can. A one-node arbiter passes the same checks and fails differently — it turns "two sites, no quorum" into "one small box, no quorum".

VERDICT

Buy the third site or give up the single namespace. Those are the two honest choices; there is no third.

05

QUORUM — WHAT A WRITE IS

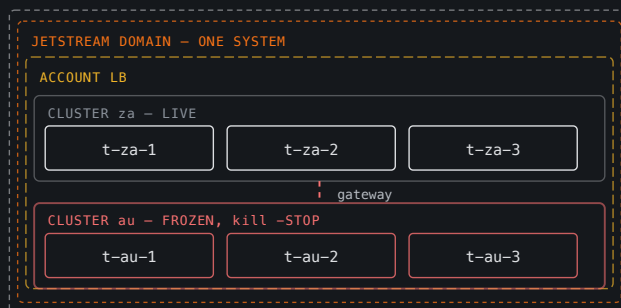
"Can we still write?" is three questions

People ask it as one question and get a wrong answer. In a gateway shape with a region dark, **one of the three works and two do not**. Which one you meant decides the whole design.

DECISION

Before you accept any topology, say **which of the three you need during an outage**. Losing the meta group is a **change** freeze, not a **write** freeze. They are not the same outage.

ONE RIG, THREE ANSWERS — T2 GATEWAY, AU FROZEN



a plain nats pub is a CORE publish. it exits 0 whether the message was stored or not.
the only proof is counting the stream afterwards — A12 F21 D23 G12 H16

THE ONE QUESTION, ASKED THREE WAYS

1 — CREATE A STREAM

needs a majority of the whole meta group
AU dark: fails, 10008 JSclusterNotAvailErr — A11

2 — PUBLISH INTO A STREAM ALREADY HERE

needs only that one stream's own replicas
AU dark: 1 message stored — A12

3 — READ THE DARK REGION'S STREAM

its replicas are all in the dark region
AU dark: fails — A21

MECHANISM

Creating

a stream is a change to the system catalogue. It is written by the meta group and needs a majority of all servers in the system.

Publishing

into an existing stream is written by that stream's own replica group. Those replicas sit inside one cluster, so a whole other region can be dark and they still have themselves.

Reading

a stream that lives in the dark region is a third thing again. Nothing there is answering, so nothing is served.

This split is why

D03-R1

in the report is three columns, not one.

THE THREE QUESTIONS AGAINST EVERY SHAPE MEASURED

SHAPE	CREATE, WITH A REGION DARK	PUBLISH INTO A LOCAL STREAM	READ THE DARK REGION
T2 / A gateway	no — 10008 A11	yes A12	no A21
T3 / C + 1-node arbiter	yes C4 C6	not measured	not measured
T4 / F + 3-node arbiter	yes F4 F23	yes F21 F24	not measured
T5 / D hub and leaf	yes D25	yes D26	not measured
T6 / G both links at once	no — 10008 G11	yes G12	not measured

PRO — OF ASKING IT THREE WAYS

You learn that a gateway outage is a **change freeze**, not a full stop.

You can size the risk properly: creation stops, existing traffic carries on.

It tells you exactly which checks to demand of any claim somebody makes.

CON — THE CATCH IN ROW TWO

"Publish keeps working" holds on **one condition**: that stream's own replicas must still hold their own majority.

Every outage here darkens a whole region, so the surviving streams kept all three replicas at home.

Losing a single stream's own replicas is a different question, and is not measured here.

06

COPYING — MIRRORS

A mirror is the second copy — and it can copy the wrong stream

Once each region has its own domain, nothing is shared until you write a mirror. The mirror must say **which domain it is reading from**. If it does not, it still works — on the wrong stream.

DECISION

Every cross-domain mirror sets `mirror.external.api`. Treat a mirror with no `external` block as a **bug**, even when it is green.

THE SAME CONFIG, ONE LINE APART

THE RIGHT WAY — NAME THE OTHER DOMAIN'S API

JETSTREAM DOMAIN za

ACCOUNT LB

MIRROR_AU

JETSTREAM DOMAIN au

ACCOUNT LB

ODOMETER

mirror.external.api = \$JS.au.API — messages copied across the domain and counted — D8 H11

THE TRAP — A NAME ALONE CANNOT CROSS AN ACCOUNT

ACCOUNT LB_AU — THE ASKER

ODOMETER — 0 msgs

MIRROR — 0 msgs

ACCEPTED. IT COPIED THE LOCAL, EMPTY ONE.

ACCOUNT LB_ZA — THE OWNER

ODOMETER — 1 msg

NOTHING LEFT THIS ACCOUNT

the mirror was NOT refused, and copied 0 of 1 messages — E17 E18 E18a

MECHANISM

- › A mirror is a stream that continuously copies another stream. It is one-way and read-only on the copy side.
- › A bare `mirror.name` is resolved **inside the asking side** — its own domain and its own account. Card 03 is the same fact seen from the other end.
- › `mirror.external.api = $JS.au.API` tells the local domain where to ask. That is the whole crossing — one line, no gateway, no shared vote.
- › The lab proves the crossing by **counting messages on both sides** (`D8` , `H11`), not by the mirror reporting healthy — the wrong mirror reports healthy too (`E18a`).

PRO

- › **Sharing without merging.** Each region keeps its own vote and still sees the other's data.
- › It is explicit. You can read the config and say what crosses.
- › It survives the source going dark — the copy is already local.

CON

- › **It is eventually consistent.** The copy lags, and during an outage it stops advancing.
- › You pay for the data twice, and for the link.
- › A misconfigured mirror is **silently wrong**, not loudly broken — it is accepted and copies 0 of 1 messages (`E17` , `E18`).

THE ONE TEST THAT CATCHES IT

- › Publish a message into the **source** region only. Then count messages in the mirror. If the count does not move, the mirror is bound to the wrong stream. Green health is not evidence.

VERDICT

A mirror is the price of a leaf-link shape. Write it explicitly, and test it by counting, never by looking at its status.

07

COPYING — VARIANT E

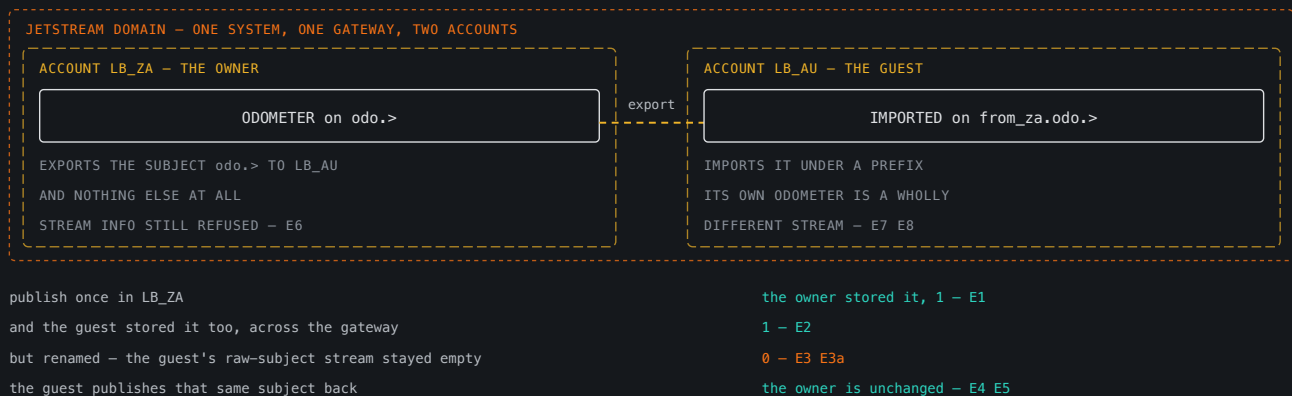
Export / import is the only sharing that is safe by design

An account shares nothing by accident. To let one account see another's messages, the owner **exports** and the guest **imports** — two edits, in two places, by two people.

DECISION

Share between tenants with **export / import** and an **import prefix**. Never by putting two tenants in one account, and never by hoping a domain will keep them apart.

ONE PUBLISH, AND WHERE IT LANDS



MECHANISM

- › The owner declares an **export** naming exactly the subjects it is willing to share. Nothing else is offered.
- › The guest declares an **import** and gives it a **prefix**. The message arrives on `from_zo.odo.>`, not on `odo.>`.
- › The prefix is the safety feature: the guest's own traffic and the imported traffic can never be confused, because they are on different subjects (`E3` , `E3a`).
- › Sharing a **subject** is not sharing a **stream**. The guest still cannot ask about the owner's stream at all (`E6`), and its own same-named stream is unrelated (`E7` , `E8`).

PRO

- › **Nothing crosses unless two people agree.** The owner offers, the guest accepts.
- › The share is narrow. One subject pattern, not an account.
- › The prefix makes the origin of every message visible in its subject.

CON

- › **It is two edits in two configs.** Get either half wrong and it silently shares nothing.
- › It shares live subjects, not history. A new guest sees nothing that happened before it imported.
- › Subject renaming must be handled by every consumer on the guest side.

WHY THIS CARD SITS NEXT TO CARD 06

- › Card 06 shows a mirror **failing silently** across an account wall (`E17` , `E18`). This card is the supported way to do what that mirror was trying to do. If you want data across an account, export it.

VERDICT

Export / import is the only account crossing that exists. It is deliberately small, and that is the point.

08

TRAPS — T6 / G

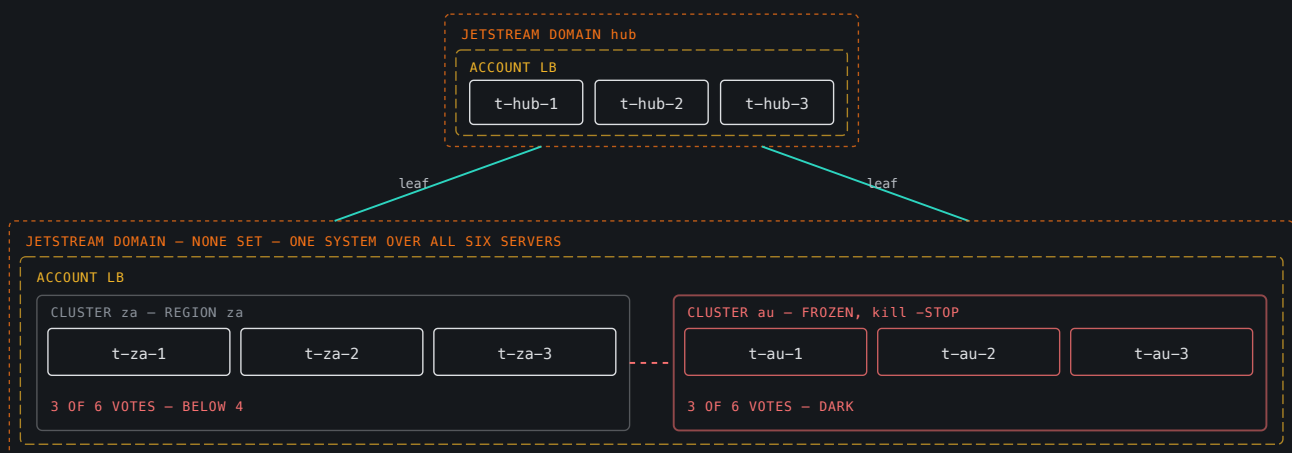
Both links at once is not both shapes at once

Adding a hub leaf link to a gateway looks like belt and braces. It is not. **The gateway decides.** You pay for the hub and the two regions get nothing back.

DECISION

Pick **one** link type between the two regions. If they are gateway-joined, a hub cannot rescue them, however healthy it is.

A HEALTHY HUB, AND A REGION THAT STILL CANNOT VOTE



healthy - the hub is one system; ZA and AU together are a second

AU dark - can ZA name a meta leader at all?

AU dark - ZA creates a new stream

AU dark - publish into ZA's existing stream

meanwhile the hub is healthy and still creates streams

2 meta groups - G1 G2 G3

NONE - G10

fails, 10008 - G11 G11a

2 messages stored - G12

and it lends ZA nothing - G13 G14 G14a

CONTINUES

The teal lines are real and the blue line still decides. **A vote cannot cross a gateway from a leaf.**

MECHANISM

- › The gateway between **za** and **au** still fuses them into **one** JetStream system of six servers. The leaf links do not undo that.
- › The hub is a **separate** system with its own meta group of three (**G1** , **G2**). A separate system has no vote in yours.
- › With AU dark, ZA holds 3 of 6 and cannot reach 4. There is **no meta leader at all** (**G10**), so creation fails with **10008** .
- › Killing the whole hub changes nothing for either region: they still see a group of 6 and both still create streams (**G16** – **G18**).

PRO — THE HONEST SHORT LIST

- › A hub gives you **one extra place to attach clients** that is not either region.
- › The hub keeps a working JetStream system of its own, and can be killed outright with no effect on either region (**G16**).

CON

- › **Every property the leaf link gave the two regions on its own is gone.** No own vote, no own namespace, no survival.
- › You pay for a third site and it is **unable to lend a vote** (**G14a**).
- › It reads as safer than a plain gateway on a diagram. It is not. That is the trap.

IF YOU WANTED A THIRD SITE TO HELP

- › Join it with a **gateway**, not a leaf link — that is card 04, and it works. A third site only votes if it is in the same system.

VERDICT

This is the first shape in the lab that costs real money and buys the two regions no quorum they did not already have. Do not build it.

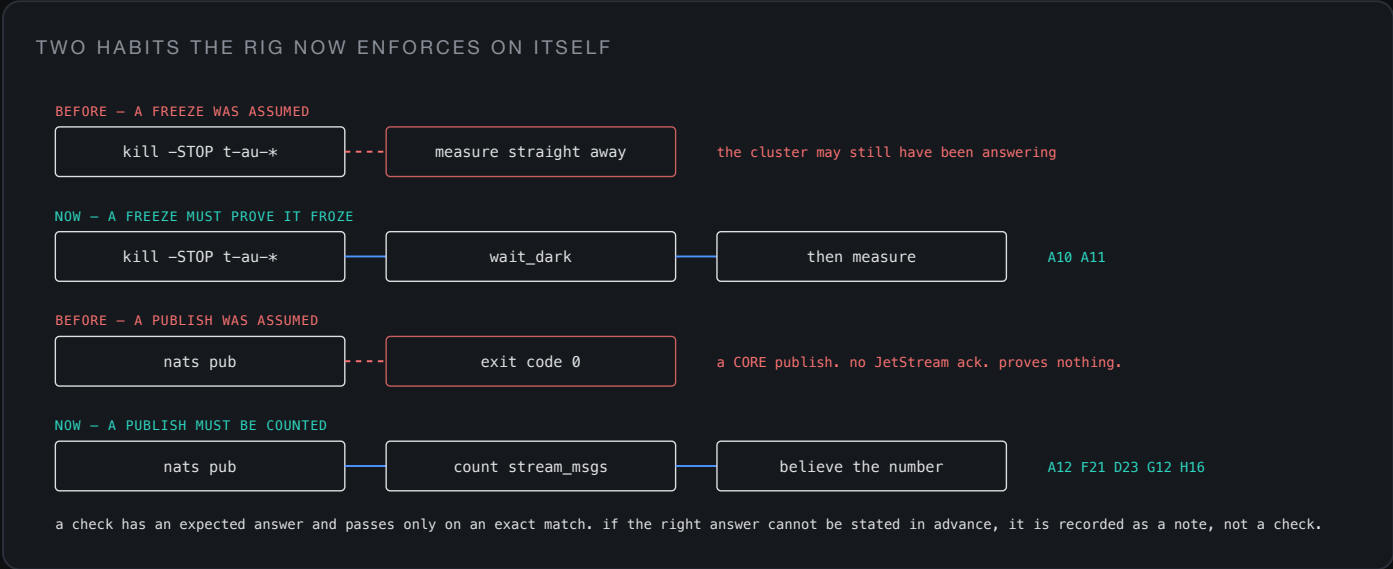
09

RETRACTIONS — THE METHOD

What this lab got wrong, and corrected

A lab whose numbers only ever improve is not measuring. Three things here were wrong, were found, and were fixed. **The fixes are now part of the rig**, so they cannot come back.

DECISION Trust a measurement only when the rig **proved the precondition** and **counted the result**. An exit code of 0 is not a measurement. A green health field is not a measurement.



CONTINUES

Both red lanes were once the real rig. **Both are now impossible to run.**

THE FOUR CORRECTIONS, AND WHAT EACH ONE BECAME

WHAT WAS WRONG

A freeze was assumed. `kill -STOP` was sent and the measurement taken at once. A stopped process can still look alive for a moment.

A publish was read from an exit code. `nats pub` exiting 0 was taken as "stored". It is a core publish and exits 0 either way.

Four summary cells claimed an answer nobody measured. Only `A21` ever reads a dark region's stream.

A run could fail and still exit 0. `check()` records a FAIL row and still returns 0.

WHAT THE RIG DOES NOW

`wait_dark` polls until the region really stops answering, and the run fails if it never does. Every outage check sits behind it.

Every publish claim is now a **message count on the stream** — `A12` , `F21` , `D23` , `G12` , `H16` .

Those four cells read **not measured**. Three sentences were corrected with them: the replica condition on "publish keeps working", that a **domain is not a wall**, and that T6's hub is healthy but cannot lend a vote.

`run-all.sh` counts FAIL rows in `results.tsv` and exits non-zero if there are any, naming the file to look in.

PRO — WHAT THE CORRECTIONS BOUGHT

- › Every remaining claim is **reproducible from nothing** by one script.
- › A wrong result now stops the run instead of printing green.
- › The four retracted cells are the reason to trust the rest of the table.

CON — WHAT THEY COST

- › Some questions lost their answer and now read **not measured**. That is honest and it is still a gap.
- › `wait_dark` and message counting make every run slower.
- › Anything quoted from before this pass may be wrong. Re-check it against `REPORT.md` .

VERDICT

Every correction here became a rule in the rig, not a note in a document. A rule in a document is a thing you forget.



CHOOSING

Pick the shape from the outage you must survive

Start from the sentence "during a region outage we must still be able to ...". The shape falls out of it. Nothing else in this deck decides for you.

THE FIVE SHAPES MEASURED, SIDE BY SIDE

SHAPE	SYSTEMS	NAMESPACE	REGION ALONE SURVIVES?	SITES NEEDED	CARD
T2 / A gateway	one	one, global	no — cannot create	2	01
T3 / C + 1-node arbiter	one	one, global	yes	3, one tiny	04
T4 / F + 3-node arbiter	one	one, global	yes	3	04
T5 / D hub and leaf	three	one per domain	yes	2 + a small hub	02
T6 / G gateway AND leaf	two	one, global	no — and you paid for a hub	3	08

CHOOSE THE LEAF SHAPE (CARD 02) WHEN

- › A region must keep taking work while the other is dark. This is the usual answer for logistics.
- › You accept a mirror per shared stream, and can name what is shared.
- › You have no credible third site, or do not want to pay for one.

CHOOSE THE ARBITER SHAPE (CARD 04) WHEN

- › One global stream name is worth more than regional independence.
- › You genuinely have a third location, not a third zone in one region.
- › You can live with every stream change crossing the ocean.

RULES THAT HOLD WHICHEVER YOU PICK

- › **Accounts for tenants, always.** A domain is not a wall (card 03).
- › **Cross an account only with export / import**, with a prefix (card 07). A mirror cannot do it and will not say so (card 06).
- › **Never add a leaf hub to a gateway pair** expecting quorum (card 08).
- › **Say which of the three writes you mean** before agreeing any of this (card 05).



PROVENANCE

Where every number came from

A figure with no provenance becomes a stale constant. Every number in this deck comes from one run of one script, on one machine, on one day.

THE RUN	
WHAT	VALUE
When	2026-09-23 09:24 SAST
Server	nats-server v2.14.6
Client	nats CLI 0.4.0
Machine	Darwin 25.4.0 arm64, all servers on one host
Result	175 checks passed, 0 failed, 51 recorded observations
Produced by	demos/03-multi-cluster-and-accounts/lab/run-all.sh
Written to	REPORT.md and REPORT.html — both generated, never hand-edited

WHICH CARD USED WHICH CHECKS		
CARD	SHAPE	CHECKS DRAWN ON THE FIGURE
01 gateway	T2 / A	A3 A4 A5 A6 – A8 A11 A12 A21
02 leaf link	T5 / D	D8 D21 D22 D24 D25 D26
03 account vs domain	T5 / D, T5 / H	D17 D18 H7 D8 H11
04 arbiter	T4 / F	F3 F4 F9 F21 F23 F24
05 three questions	T2 / A	A11 A12 A21 , table adds C4 C6 D25 D26 F4 F23 G11 G12
06 mirrors	T5 / D, variant E	D8 H11 E17 E18 E18a
07 export / import	variant E	E1 – E8
08 both links	T6 / G	G1 – G3 G10 – G14a G16 – G18
09 retractions	method	A10 A11 A12 F21 D23 G12 H16

WHAT THESE NUMBERS ARE NOT

>

No timing on this page is a production figure. Every server ran on one laptop with no WAN between them. Failover times measured here mean nothing for a real deployment.

>

Every outage is kill -STOP on a whole region. No split brain, no packet loss, no partial failure, no single-server loss inside a live cluster.

>

A check ID is permanent. IDs are never reused or renumbered, so a card citing A11 keeps meaning the same measurement across runs.

>

If a card and REPORT.md disagree, the report is right. Re-run lab/run-all.sh and correct the card.